

ReviewAgent

An Agentic Pipeline for App Review Triage,
Resolution, and Response Generation

Research Proposal

Prepared by: Fabiha Jalal

Date: March 2026

1. Research Questions

RQ1: Review-to-Issue Translation Quality

How accurately can an LLM-based agent translate noisy, unstructured app review clusters into structured, taxonomy-grounded issue specifications, and how do these compare to human-written GitHub issues in terms of completeness, accuracy, actionability, specificity, and clarity?

RQ2: Coupled Resolution and Response Generation

Does coupling knowledge-graph-based issue prioritization with agentic code resolution and resolution-aware response generation produce more specific, actionable, and helpful user responses compared to context-unaware baselines (e.g., RRGGen, CoRe)?

RQ3: Dual-Objective RLHF Effectiveness

Does dual-objective RLHF (optimizing for both response quality and policy compliance) with dimension-level expert feedback outperform single-objective preference tuning in generating safe, helpful, and accurate app review responses over iterative retraining cycles?

2. Research Objective and Specific Aims

Objective

To design, implement, and evaluate an end-to-end agentic pipeline, ReviewAgent, that automatically triages noisy app store reviews, translates them into structured and actionable issue specifications, resolves them through multi-agent code repair, and generates resolution-aware user responses, all refined through human-in-the-loop feedback at critical decision points.

Aim 1: Develop a Unified Clustering-to-Issue Translation Framework

Design a three-layer framework (knowledge graph construction, hierarchical clustering, and standardized schema mapping) that converts raw app reviews into structured, taxonomy-grounded issue specifications covering bug reports, feature requests, performance complaints, usability issues, and compatibility issues. Validate output quality using a 5-dimension expert rubric with inter-annotator agreement.

Aim 2: Build a Coupled Resolution-and-Response System

Implement a multi-agent code resolution pipeline (Planner, Navigator, Editor, Executor) that consumes validated issue specs, and a parallel response generation module that is aware of the resolution outcome, producing user-facing replies that reference the specific issue and its fix status rather than generic templates.

Aim 3: Design and Evaluate a Dual-Objective RLHF Loop

Embed human oversight at three pipeline stages (classification verification, issue spec validation, final response review) and implement a progressive RLHF strategy (KTO to DPO to Constrained PPO) that jointly optimizes for response helpfulness and policy compliance, with feedback propagating backward to improve upstream stages.

3. Possible Outcomes of the Research

- 1. Novel Benchmark Dataset:** The first dataset pairing app review clusters with expert-written structured issue specifications (200 to 300 clusters, 3+ annotators), serving as a reusable evaluation resource for the research community.
- 2. End-to-End Resolution Feasibility:** Empirical evidence (via RQ1 to RQ3) demonstrating that the pipeline can convert raw reviews into validated patches and specific user responses, with measurable improvements over standalone baselines (BERT-RCNN for classification, CoRe/RRGen for response generation, SWE-Agent for code repair).
- 3. Dual-Objective RLHF Superiority:** Evidence that decomposing feedback into quality and compliance dimensions, scored across 5 rubric criteria, produces safer and more helpful responses than binary preference signals alone.
- 4. Triage Effort Reduction:** Quantified reduction in developer triage effort through KG-prioritized issue clusters compared to raw review feeds, measured via time-to-first-action and triage accuracy.
- 5. Multiple Publication Contributions:** The full system targets ICSE/FSE, the translation component and dataset targets ASE, the RLHF application targets EMNLP, and the KG-based triage targets the RE conference.

4. Proposed Methodological Steps and Architecture

4.1 System Architecture Overview

The ReviewAgent pipeline consists of five stages with human-in-the-loop (HITL) checkpoints integrated at three critical junctures. The architecture is designed so that human experts act as quality gates, catching errors early before they propagate downstream.

Stage 1: Intake with Human Verification

Raw app reviews undergo three parallel NLP tasks: (1) multi-label classification using a fine-tuned RoBERTa model trained on the MAALEJ and GUZMAN datasets, (2) aspect-based sentiment analysis to capture per-aspect polarity (e.g., 'Great UI but terrible battery drain' yields UI:positive, battery:negative), and (3) entity extraction to identify devices, OS versions, screens, and feature names. Ambiguous classifications (below a confidence threshold) are flagged for expert verification, and corrections feed back into the training loop.

Stage 2: Unified Clustering-to-Issue Framework

This stage implements a three-layer process:

1. Knowledge Graph Construction: Structured review objects become nodes (aspects, entities, reviews) and edges (sentiment, temporal) in a knowledge graph.
2. Hierarchical Clustering: Reviews are grouped first by aspect (coarse), then sub-clustered by specific complaint using semantic similarity and graph structure.
3. Standardized Issue Schema Mapping: Each cluster is mapped to a fixed schema containing fields for issue type, affected component, sentiment distribution, representative reviews, entities, temporal patterns, and a priority score computed via graph centrality (PageRank/betweenness).

Stage 3: Review-to-Issue Translation with Expert Validation

The core novel contribution. An LLM agent receives standardized cluster schemas and generates GitHub-issue-quality specifications grounded in established SE taxonomies:

1. Bug reports follow the Zimmermann template (summary, steps to reproduce, expected vs. actual behavior, environment, severity)
2. Feature requests use user story format (As a [user], I want [feature], so that [benefit])
3. Performance complaints map to non-functional requirement categories (latency, memory, battery drain, network)
4. Usability issues align with Nielsen's heuristics (visibility, feedback, error prevention)
5. Compatibility issues generate device-OS-version matrices

Domain experts review each generated issue spec using a standardized 5-dimension rubric (completeness, accuracy, actionability, specificity, clarity) scored 1 to 5. Multiple experts rate independently; inter-annotator agreement is measured via Cohen's kappa or Krippendorff's alpha. Low-scoring specs are returned to the LLM with dimension-level feedback for regeneration.

Stage 4a: Agentic Code Resolution

A multi-agent system receives validated issue specs and operates as a software engineering team:

1. Planner Agent: Reads the issue spec and codebase, creates a resolution strategy, and breaks the fix into subtasks.
2. Navigator Agent: Searches the codebase using affected component fields, identifies relevant files and functions, and checks KG context for related issues.
3. Editor Agent: Writes the code fix based on the Planner's strategy and Navigator's findings.
4. Executor Agent: Runs the test suite, generates new test cases for the fix, and loops back to the Editor on failure. Produces a validated patch with a test report.

Stage 4b: Resolution-Aware Response Generation

Running in parallel with Stage 4a, this module generates user-facing responses using Retrieval-Augmented Generation (RAG) to pull past responses, FAQs, and changelogs. Responses are resolution-aware: they reference the specific issue and fix status from Stage 4a. A self-refinement loop (2 to 3 iterations) critiques and improves the draft for vagueness, unauthorized promises, and empathy before producing the final output.

Stage 5: Dual-Objective Human Feedback

Experts provide dimension-level feedback across two streams:

Stream 1 (Quality): Helpfulness, specificity, empathy, accuracy, and actionability.

Stream 2 (Policy Compliance): Promise safety, information leakage, tone guidelines, and legal safety.

Feedback propagates backward: corrections improve Stage 1 classification, rubric scores refine Stage 3 prompting/fine-tuning, and response quality scores drive the RLHF training loop progressing from KTO (binary signals, small data) to DPO (paired preferences) to Constrained PPO (maximize helpfulness subject to compliance constraints) at scale.

4.2 Methodological Steps

Step	Activity	Output
Step 1	Collect and preprocess datasets (RRGen 570K, MAALEJ 4K, GUZMAN); identify open-source Android apps with both Play Store reviews and GitHub repos	Curated multi-source dataset
Step 2	Fine-tune RoBERTa for multi-label review classification; implement aspect-based sentiment analysis and entity extraction with HITL verification	Stage 1 classifier with active learning loop
Step 3	Build the knowledge graph; implement hierarchical clustering and standardized schema mapping with priority ranking	Unified clustering-to-issue framework
Step 4	Develop LLM-based review-to-issue translation agent with taxonomy-grounded prompting; construct gold-standard dataset (200 to 300 clusters, 3+ expert annotators)	Issue translation module + benchmark dataset
Step 5	Implement multi-agent code resolution system (Planner, Navigator, Editor, Executor) and resolution-aware response generation with RAG	Stages 4a and 4b
Step 6	Build dual-objective RLHF pipeline (KTO, DPO, Constrained PPO) with backward feedback propagation across all stages	Stage 5 + iterative improvement loop
Step 7	End-to-end evaluation against baselines (BERT-RCNN, CoRe, SWE-Agent); ablation studies per stage; expert rubric evaluation with inter-annotator agreement	Experimental results for RQ1 to RQ3

4.3 Dataset Strategy

Dataset	What It Provides	Pipeline Stage
RRGen (~570K pairs)	Google Play review-response pairs for training response generation	Stages 1, 4b
MAALEJ (~4K labeled)	Multi-label classified app reviews (bug, feature, rating, UX)	Stage 1
GUZMAN	Aspect-level annotations for app reviews	Stage 1
Open-source Android apps with GitHub repos	Ground-truth mapping: reviews to GitHub issues to code fixes	Stages 3, 4a
Custom gold-standard (to be built)	Expert-written structured issue specs paired with review clusters	Stage 3 evaluation
SWE-bench	Benchmark for evaluating agentic code resolution quality	Stage 4a

4.4 Evaluation Metrics

Stage	Metric	Baseline
Stage 1: Intake	F1 score (multi-label classification)	BERT-RCNN
Stage 2: Clustering	Cluster purity, deduplication precision	Standard clustering
Stage 3: Translation	Expert rubric scores (5 dimensions, 1 to 5), inter-annotator agreement	Human-written issues
Stage 4a: Resolution	Patch correctness (SWE-bench style)	SWE-Agent
Stage 4b: Response	BLEU + human evaluation (helpfulness, specificity)	CoRe, RRGGen
Stage 5: RLHF	Preference win rate over iterations	Single-objective tuning